Actividad 1

Regresión lineal

Autor: Jesús Emiliano Hernández Montalvo

```
# Importa librerías necesarias
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from scipy import stats
import pandas as pd
```

```
# Semilla random establecida
np.random.seed(38)

# Parámetros iniciales
n = 100 # número de observaciones

# Generar valores de x entre 0 y 10
x = np.random.rand(n, 1) * 10

# Ruido de la función
ruido = np.random.randn(n, 1) * 2 # con desviación de 2

# Calcular y usando la fórmula y = 2x + 3 + ε, donde ε ~ N(0,1.5)
y = 2 * x + 3 + ruido

# Crear un Dataframe
df = pd.DataFrame({'x': x.flatten(), 'y': y.flatten()})

# Mostrar las primeras 5 filas del Dataframe
display(df)
```

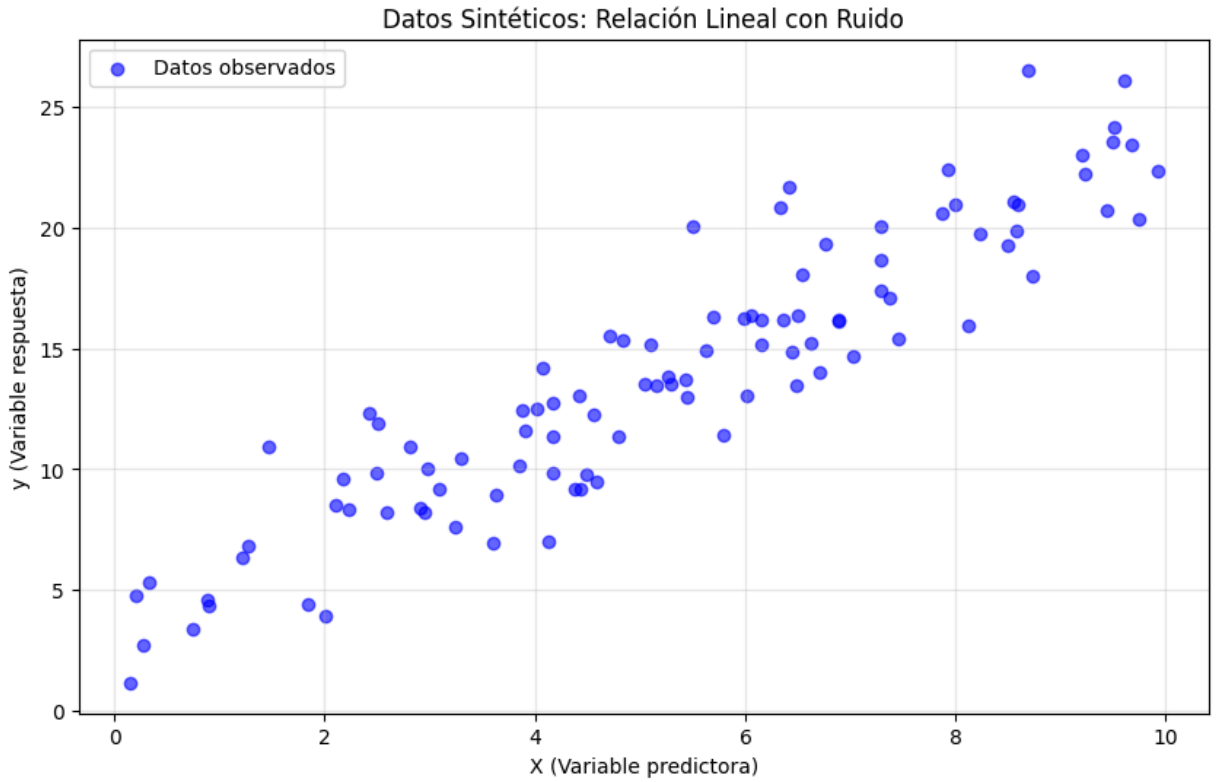
	x	y
0	3.847731	10.171440
1	8.597078	20.974718
2	9.441996	20.707291
3	7.028249	14.689797
4	6.336341	20.808754
...	...	...
95	6.769389	19.297371
96	5.101022	15.147510
97	2.493281	9.862291
98	1.473121	10.927588
99	4.076523	14.166444

100 rows × 2 columns

Próximos pasos: Generar código con df | New interactive sheet

```
# Generar una visualización exploratoria de los datos
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, color='blue', label='Datos observados')
plt.xlabel('X (Variable predictora)')
plt.ylabel('y (Variable respuesta)')
plt.title('Datos Sintéticos: Relación Lineal con Ruido')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

# Estadísticas descriptivas
print(f"Media de X: {np.mean(x):.3f}")
print(f"Media de y: {np.mean(y):.3f}")
print(f"Desviación estándar de X: {np.std(x):.3f}")
print(f"Desviación estándar de y: {np.std(y):.3f}")
print(f"Correlación entre X e y: {np.corrcoef(x.flatten(), y.flatten())[0,1]:.4
```




```
=== Estadísticas Descriptivas ===
Media de X: 5.239
Media de y: 13.876
Desviación estándar de X: 2.582
Desviación estándar de y: 5.682
```

```
# Calcular la varianza de cada una de las columnas
variance = df.var()

# Mostrar la varianza
display(variance)
```

	0
x	6.732364
y	32.616092

dtype: float64

```
# Calcular la varianza de 'x'
x_mean = x.mean()
x_variance_formula = ((df['x'] - x_mean)**2).sum() / (N - 1)

# Calcular la varianza de 'y'
y_mean = y.mean()
y_variance_formula = ((df['y'] - y_mean)**2).sum() / (N - 1)

# Mostrar resultados
```

```
print(f"Varianza de 'x' usando la fórmula: {x_variance_formula:.6f}")
print(f"Varianza de 'y' usando la fórmula: {y_variance_formula:.6f}")
```

```
Varianza de 'x' usando la fórmula: 6.732364
Varianza de 'y' usando la fórmula: 32.616092
```

```
# Calcular el numerador de b1: sum((xi - x_mean) * (yi - y_mean))
Sxy = ((x - x_mean) * (y- y_mean)).sum()

# Calcular el denominador de b1: sum((xi - x_mean)^2)
Sxx = ((x - x_mean)**2).sum()

# Calcular b1
b1 = Sxy / Sxx

# Calcular b0
b0 = y_mean - b1 * x_mean

# Convertir a escalares si son arrays
if hasattr(b0, "__len__"):
    b0 = b0[0]
if hasattr(b1, "__len__"):
    b1 = b1[0]


# Mostrar coeficientes
print(f"Coefficient b1 (pendiente): {b1:.6f}")
print(f"Coefficient b0 (intersección): {b0:.6f}")
print("== Valores reales ==")
print(f"Coefficient b1 (pendiente): {2:.6f}")
print(f"Coefficient b0 (intersección): {3:.6f}")
```

```
Coefficient b1 (pendiente): 2.035032
Coefficient b0 (intersección): 3.214861
== Valores reales ==
Coefficient b1 (pendiente): 2.000000
Coefficient b0 (intersección): 3.000000
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Calcular las 'y' predecidas usando los escalares
y_pred = b0 + b1 * x

# Calculo de residuos
residuo = y - y_pred

# Mostrar los valores
df['y_pred'] = y_pred
```

```
df['residuo'] = residuo
display(df)
```

	x	y	y_pred	residuo
0	3.847731	10.171440	11.045119	-0.873679
1	8.597078	20.974718	20.710195	0.264523
2	9.441996	20.707291	22.429630	-1.722339
3	7.028249	14.689797	17.517576	-2.827779
4	6.336341	20.808754	16.109521	4.699233
...	...	...	...	...
95	6.769389	19.297371	16.990788	2.306583
96	5.101022	15.147510	13.595606	1.551905
97	2.493281	9.862291	8.288770	1.573521
98	1.473121	10.927588	6.212710	4.714878
99	4.076523	14.166444	11.510718	2.655726

100 rows × 4 columns

Próximos pasos: Generar código con df | New interactive sheet

```
# Estadísticos de los resiudos
print("== Estadísticos de los residuos ==")
print(f"Media de residuos: {np.mean(residuo):.6f} (debe ser ~0)")
print(f"Varianza de residuos: {np.var(residuo):.6f}")
print(f"Desviación estándar de residuos: {np.std(residuo):.4f}")
print(f"Suma de residuos: {np.sum(residuo):.6f} (debe ser ~0)")
```

```
== Estadísticos de los residuos ==
Media de residuos: 0.000000 (debe ser ~0)
Varianza de residuos: 4.687618
Desviación estándar de residuos: 2.1651
Suma de residuos: 0.000000 (debe ser ~0)
```

```
SST = np.sum((df['y'] - np.mean(y))**2)
SSE = np.sum((df['y'] - (b0 + b1 * df['x']))**2)
SSR = SST - SSE

print(f"SST: {SST:.6f}")
print(f"SSE: {SSE:.6f}")
print(f"SSR: {SSR:.6f}")

# cálculo de métricas de calidad de modelo
R2 = SSR / SST
```

```
print(f"R-squared (R2): {R2:.6f}")

#MSE
MSE = np.mean((df['y'] - (b0 + b1 * df['x']))**2)
print(f"Error cuadrático promedio (MSE): {MSE:.6f}")

# RMSE
RMSE = np.sqrt(MSE)
print(f"RRaíz del error cuadrático promedio (RMSE): {RMSE:.6f}")

# error absoluto medio
MAE = np.mean(np.abs(df['y'] - (b0 + b1 * df['x'])))
print(f"Error absoluto medio (MAE): {MAE:.6f}")

MAPE= np.mean(np.abs((df['y'] - (b0 + b1 * df['x'])) / df['y'])) * 100
print(f"Error porcentual absoluto medio (MAPE): {MAPE:.6f}")
```

```
SST: 3228.993085
SSE: 468.761825
SSR: 2760.231260
R-squared (R2): 0.854827
Error cuadrático promedio (MSE): 4.687618
RRaíz del error cuadrático promedio (RMSE): 2.165091
Error absoluto medio (MAE): 1.701761
Error porcentual absoluto medio (MAPE): 16.739404
```

```
# Grados de libertad
n = len(y)
p = 1 # Número de variables predictorias
df_error = n-p-1 #grados de libertad del error

# Varianza del error (estimador sin sesgo)
sigma2_estimado = SSE / df_error
sigma_estimado = np.sqrt(sigma2_estimado)

# Varianza del error (estimador sesgado para realizar una comparación)
sigma2_sesgado = SSE / n

print(f"\n=== Grados de Libertad ===")
print(f"n (observaciones): {n}")
print(f"p (predictores): {p}")
print(f"df_error (n-p-1): {df_error}")

print(f"\n=== Varianza del Error ===")
print(f"σ² estimado (insesgado) = SSE/(n-p-1) = {sigma2_estimado:.6f}")
print(f"σ² sesgado = SSE/n = {sigma2_sesgado:.6f}")
print(f"σ estimado = √σ² = {sigma_estimado:.6f}")

print(f"\n=== Comparación con valor real ===")
print(f"σ² real = {2**2:.4f} (ruido con desviación 2)")
print(f"σ² estimado = {sigma2_estimado:.4f}")
print(f"Diferencia: {abs(sigma2_estimado - 2**2):.4f}")
```

```
=== Grados de Libertad ===
n (observaciones): 100
p (predictores): 1
df_error (n-p-1): 98

=== Varianza del Error ===
σ² estimado (insesgado) = SSE/(n-p-1) = 4.783284
σ² sesgado = SSE/n = 4.687618
σ estimado = √σ² = 2.187072

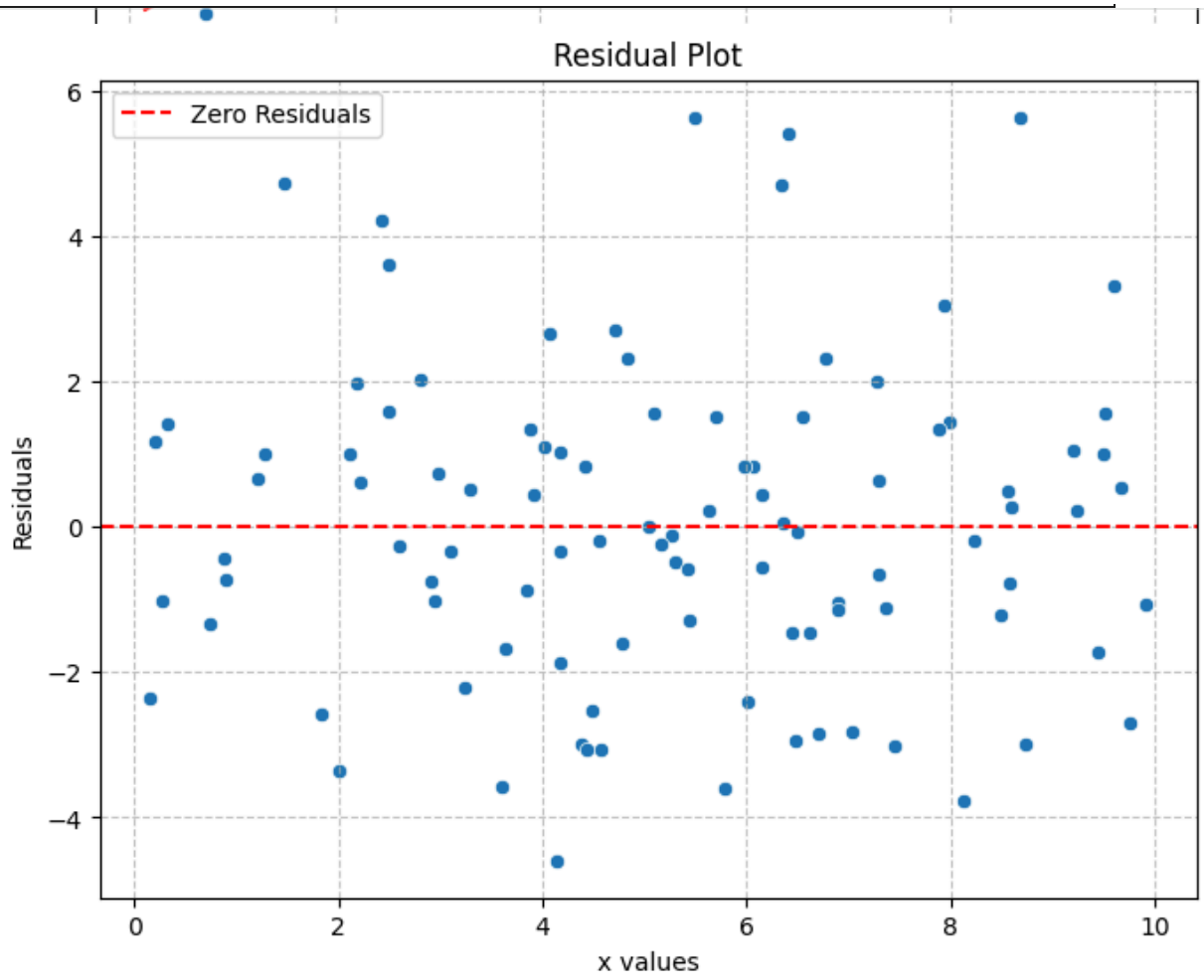
=== Comparación con valor real ===
σ² real = 4.0000 (ruido con desviación 2)
σ² estimado = 4.7833
Diferencia: 0.7833
```

```
# Crear un scatter plot
plt.figure(figsize=(8, 6))
sns.scatterplot(x='x', y='y', data=df, label='Actual Data')
sns.lineplot(x='x', y='y_pred', data=df, color='red', label='Regression Line')
plt.title('Scatter Plot with Regression Line')
plt.xlabel('x values')
plt.ylabel('y values')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.show()
```

```
# Calcular residuos
df['residuals'] = df['y'] - df['y_pred']

# Crear un residual plot
plt.figure(figsize=(8, 6))
sns.scatterplot(x='x', y='residuals', data=df)
plt.axhline(y=0, color='red', linestyle='--', label='Zero Residuals') #
plt.title('Residual Plot')
plt.xlabel('x values')
plt.ylabel('Residuals')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.show()

# Análisis de residuos
print("=== Análisis de Residuos ===")
print("✓ Los residuos deben distribuirse aleatoriamente alrededor de 0"
print("✓ No debe haber forma de embudo (heterocedasticidad)")
print("✓ No debe haber curvatura (no linealidad)")
```




```
=== Análisis de Residuos ===
✓ Los residuos deben distribuirse aleatoriamente alrededor de 0
✓ No debe haber forma de embudo (heterocedasticidad)
```